

Documentation : Projet Yams - Interface, Simulations et Agents IA

Cette documentation présente une application complète de jeu de Yams (Yatzy) développée en Python. Le projet intègre un moteur de jeu robuste, une interface graphique moderne et un framework permettant de faire s'affronter divers agents, allant de stratégies simples à des modèles de langage (LLM) locaux.

```
1  .
2  |— agents/
3  |   |— base.py          # Interface abstraite pour tous les agents
4  |   |— expert_agent.py  # Logique basée sur la détection
5  |   d'opportunité
6  |   |— greedy_agent.py  # Stratégie de maximisation immédiate
7  |   |— hardcoded_agent.py # Stratégie déterministe de référence
8  |   |— llm_local.py     # Intégration Ollama (phi3:mini)
9  |   |— random_agent.py  # Agent de bse (choix aléatoires)
10 |   |— assets/
11 |      |— icon.png      # Ressources graphiques
12 |      |— MonIcon.icns
13 |   |— core/
14 |      |— game.py       # Gestionnaire d'état (dés, lancers,
15 |      |   tours)
16 |      |— probabilities.py # Moteur de calcul statistique
17 |      |— scoring.py     # Règles de calcul des points
18 |      |— utils.py       # Utilitaires de détection de
19 |      combinaisons
20 |   |— scripts/
21 |      |— export_csv.py  # Outil d'exportation des données
22 |      |— simulate_csv.py # Simulation massive via CLI
23 |      |— simulate_llm_vs_hc.py # Benchmark LLM vs Hardcoded
24 |      |— simulate_llm.py # Tests isolés pour l'agent IA
25 |   |— storage/
26 |      |— sqlite.py      # Persistance des données (SQLite)
27 |   |— tournament/
28 |      |— manager.py     # Orchestration des tournois
29 |      |— results.py     # Agrégation des statistiques
30 |   |— ui/
31 |      |— agent_config_dialog.py # Configuration des participant
32 |      |— main_window.py  # Fenêtre principale de l'application
33 |      |— styles/         # Thèmes (industrial_dark.qss)
34 |      |— widgets/       # Composants graphiques (Dés, Scores, Logs)
```

```

33 |— app.py                # Point d'entrée de l'application GUI
34 |— Makefile             # Automatisation des tâches de build
35 |— requirement # Dépendances du projet
36
37

```

1. Présentation générale

L'application permet non seulement de jouer au Yams de manière classique, mais sert également de plateforme de benchmark pour l'intelligence artificielle. Elle propose des outils d'analyse probabiliste en temps réel pour guider les décisions des joueurs ou des agents.

Fonctionnalités clés

Moteur de jeu complet : Gestion des 5 dés, des trois lancers par tour et des règles de score pour toutes les catégories standard.

Framework multi-agents : Interface unifiée pour intégrer des stratégies aléatoires, gloutonnes, expertes ou basées sur l'IA.

Intégration LLM : Utilisation de modèles locaux (via Ollama et `phi3:mini`) pour la prise de décision.

Mode Tournoi : Orchestration de matchs multi-parties avec agrégation statistique des résultats.

Interface Graphique (GUI) : Fenêtre interactive avec visualisation des dés, des scores et un panneau de probabilités, le tout sous un thème sombre industriel.

2. Architecture du projet

Le projet suit une structure modulaire stricte pour garantir l'évolutivité et la séparation des responsabilités.

Structure des répertoires

core/ : Contient le cœur du moteur (logique de jeu, calcul des points, utilitaires et probabilités).

agents/ : Regroupe la classe de base abstraite et toutes les implémentations concrètes d'agents.

tournament/ : Gère l'orchestration des compétitions et l'analyse des résultats.

ui/ : Contient les composants de l'interface graphique (fenêtres, widgets personnalisés et styles).

storage/ : Gestion de la persistance des données via SQLite et export CSV.

scripts/ : Points d'entrée pour les simulations en ligne de commande (CLI).

3. Le Moteur de Jeu (core)

Le moteur est la fondation du projet, assurant le respect des règles du Yams.

Gestion de l'état (game . py)

La classe `GameState` gère le cycle de vie d'un tour :

Initialisation : Création de 5 dés (valeur 0) et remise à zéro du compteur de lancers.

Nouveau tour (new_hand) : Lance 5 dés avec des valeurs aléatoires de 1 à 6.

Relance (reroll_unheld) : Permet de relancer les dés non verrouillés par un masque de booléens, dans la limite de 3 lancers par tour.

Calcul du score (scoring . py)

Ce module implémente les règles pour les 13 catégories standard (As à Six, Brelan, Carré, Full, Suites, Yams et Chance). Les scores sont soit la somme des dés correspondants, soit des valeurs fixes (ex: 25 points pour un Full, 50 pour un Yams).

4. Les Agents et Stratégies

Tous les agents héritent d'une classe abstraite `Agent` imposant deux méthodes : `choose_hold` (quels dés garder) et `choose_category` (où inscrire le score).

Types d'agents disponibles

Types d'agents disponibles		
Agent	Stratégie	Usage
Random	Choix totalement aléatoires.	Baseline de test.
Greedy	Maximise le score immédiat et garde la valeur majoritaire.	Test de performance simple.
Expert	Détecte les opportunités (proche d'une suite) et priorise les catégories rares.	Stratégie optimisée.
Hardcoded	Suite de règles strictes et déterministes sans pondération.	Benchmark stable pour le LLM.
LLM	Envoie des prompts à un modèle local (Ollama) pour décider.	Recherche et expérimentation.

5. Interface Graphique et Simulations

L'application propose deux modes d'interaction principaux.

Interface Graphique (GUI)

Conçue avec PyQt/PySide, elle offre une expérience utilisateur riche.

DiceWidget : Affiche les dés avec leur état (gardé ou non).

ProbabilityPanel : Affiche dynamiquement les chances d'obtenir chaque combinaison selon les dés actuels.

LogView : Permet de suivre les réflexions des agents, notamment les échanges avec le LLM.

Simulations CLI

Pour des analyses massives, des scripts permettent de lancer des tournois sans interface graphique. Les résultats (victoires, scores moyens) sont agrégés et peuvent être sauvegardés en base SQLite ou exportés en CSV pour un traitement statistique ultérieur.

6. Installation et Utilisation

Prérequis

- Python 3.10+.
- Bibliothèques listées dans `requirements.txt`.

Ollama (optionnel, pour l'agent LLM) avec le modèle `phi3:mini`.

Lancement

- Interface principale : `python app.py` ou `python run.py`.
- Sélecteur d'agents : `python run_select.py`.
- Simulations massives : `python scripts/simulate_csv.py`.

Un **Makefile** et un script **build.sh** sont fournis pour automatiser le packaging et la préparation de l'environnement.